# Tema 4 — Modelos de Machine Learning No Supervisados
# Apuntes extensos con ejemplos y programación

## Índice


**1. Qué significa "no supervisado" y cómo se trabaja en la práctica** **3**
- 1.1. Objetivos típicos . . . 3
- 1.2. Flujo de trabajo recomendado (receta general) . . . 3

**2. K-Means (segmentación por centroides)** **4**
- 2.1. Idea central . . . 4
- 2.2. Algoritmo paso a paso (lo que hace realmente) . . . 4
- 2.3. Intuición (por qué funciona) . . . 4
- 2.4. Preprocesado: por qué hay que escalar . . . 4
- 2.5. Programación en scikit-learn (explicada parámetro por parámetro) . . . 5
- 2.6. Ejemplo numérico a mano (para entender "cómo asigna") . . . 5
- 2.7. Cómo elegir $K$ (y por qué el codo no es magia) . . . 5
- 2.8. Interpretación de clusters (customer segmentation) . . . 6
- 2.9. Limitaciones típicas . . . 6
- 2.10. Pregunta típica: "¿se combinan métodos o van separados?" . . . 6

**3. DBSCAN (clustering por densidad + detección de ruido)** **6**
- 3.1. Idea central . . . 6
- 3.2. Parámetros clave . . . 7
- 3.3. Algoritmo paso a paso . . . 7
- 3.4. Programación en scikit-learn . . . 7
- 3.5. Cómo elegir `eps` (técnica típica) . . . 7
- 3.6. Ejemplo numérico a mano (mini) . . . 8
- 3.7. DBSCAN en geolocalización (lat/long): el detalle importante . . . 8
- 3.8. Ventajas y limitaciones . . . 8
- 3.9. Pregunta típica: "¿cómo asigno un punto nuevo?" . . . 9

**4. HDBSCAN (densidad jerárquica, más robusto)** **9**
- 4.1. Por qué existe . . . 9
- 4.2. Parámetros clave . . . 9
- 4.3. Programación (librería hdbscan) . . . 9
- 4.4. Interpretación . . . 9
- 4.5. Ventajas y limitaciones . . . 9

**5. PCA (Análisis de Componentes Principales)** **10**
5.1. Qué es y qué NO es . . . 10
5.2. Objetivo . . . 10
5.3. Por qué hay que escalar (casi siempre) . . . 10
5.4. Programación en scikit-learn (y qué significa cada atributo) . . . 10
5.5. Elegir número de componentes . . . 10
5.6. Ejemplo numérico conceptual (2D correlacionado) . . . 11
5.7. PCA + clustering (combinación típica) . . . 11

**6. t-SNE (visualización preservando vecindades locales)** **11**
6.1. Qué hace . . . 11
6.2. Cuándo usarlo . . . 11
6.3. Lo que NO debes interpretar . . . 12
6.4. Parámetros clave y su significado . . . 12
6.5. Programación recomendada (con PCA previo) . . . 12
6.6. Cómo se usa con clustering . . . 12

**7. UMAP (embeddings rápidos y transformables)** **13**
7.1. Qué hace . . . 13
7.2. Por qué es tan usado . . . 13
7.3. Parámetros clave . . . 13
7.4. Programación . . . 13
7.5. UMAP con datos nuevos (ventaja clave) . . . 13

**8. Comparativa final (qué método usar, cuándo y por qué)** **14**

**9. Casos de uso reales (empresa / vida real) y cómo se conectan con el código** **14**
9.1. Segmentación de clientes (K-Means / HDBSCAN) . . . 14
9.2. Clustering geográfico (DBSCAN/HDBSCAN) . . . 14
9.3. Visualización de datos complejos (PCA, t-SNE, UMAP) . . . 14

**10. Checklist de examen/prácticas (lo que suele hacerte perder puntos)** **15**

**11. Resumen final ultra-conciso (por si quieres memorizar)** **15**

# 1. Qué significa "no supervisado" y cómo se trabaja en la práctica

En aprendizaje **no supervisado** normalmente no tenemos etiquetas $y$; solo contamos con una matriz de características $X \in \mathbb{R}^{n \times p}$ donde:

- $n$ = número de observaciones (clientes, museos, transacciones, etc.).
- $p$ = número de variables (edad, ingresos, latitud, longitud, variables de consumo, etc.).

## 1.1. Objetivos típicos

- **Clustering** (segmentación): agrupar observaciones similares

  Ejemplos: segmentación de clientes, agrupación de tiendas, clustering geográfico, perfiles de usuarios.

- **Reducción de dimensionalidad / embeddings**: representar datos en 2D/3D o en menos variables

  Ejemplos: visualizar datos de alta dimensión, eliminar ruido, acelerar modelos, explorar estructura.

- **Detección de anomalías**: puntos raros / outliers

  Ejemplos: fraude, fallos de sensores, clientes con comportamiento extraño.

## 1.2. Flujo de trabajo recomendado (receta general)

Este esquema te vale para casi cualquier práctica/examen:

1. **Entender variables**: tipos, escalas, unidades, valores imposibles, nulos.
2. **Limpieza**: eliminar/ imputar NaNs, tratar outliers si procede, eliminar columnas irrelevantes (IDs).
3. **Preprocesado**: escalado (`StandardScaler`) si hay distancias (K-Means, DBSCAN, UMAP, t-SNE).
4. **Elegir método**: K-Means (clusters "redondos"), DBSCAN/HDBSCAN (densidad + ruido), PCA/t-SNE/UMAP (visualización/reducción).
5. **Entrenar** (`fit` / `fit_predict`) y obtener **labels**.
6. **Evaluación** sin etiquetas: silhouette, inercia/elbow, inspección de tamaños, estabilidad, validación de negocio.
7. **Interpretación**: perfiles por cluster (medias, percentiles), reglas de negocio, acciones (marketing, riesgo, producto).

# 2. K-Means (segmentación por centroides)

## 2.1. Idea central

K-Means busca dividir los datos en $K$ grupos minimizando la suma de distancias cuadradas de cada punto a su centroide:

$$J = \sum_{i=1}^{n} \|x_i - \mu_{c(i)}\|^2$$

donde:

- $x_i$ es el punto $i$.
- $c(i)$ es el índice del cluster asignado a $x_i$.
- $\mu_k$ es el centroide (media) del cluster $k$.

## 2.2. Algoritmo paso a paso (lo que hace realmente)

1. Elegir $K$.
2. Inicializar $K$ centroides (mejor con `k-means++`).
3. **Asignación**: cada punto se asigna al centroide más cercano.
4. **Actualización**: cada centroide pasa a ser la media de los puntos asignados.
5. Repetir 3–4 hasta convergencia o `max_iter`.

## 2.3. Intuición (por qué funciona)

K-Means alterna entre:

- minimizar $J$ con centroides fijos (asignación por distancia mínima),
- minimizar $J$ con asignaciones fijas (el mejor centroide es la media).

Converge a un mínimo local: por eso importa la inicialización y `n_init`.

## 2.4. Preprocesado: por qué hay que escalar

Como K-Means usa distancias, una variable con rango grande domina. Ejemplo:

- `Income` en miles,
- `Age` en decenas.

Si no escalas, K-Means agrupa casi solo por `Income`. Solución típica:

```
from sklearn.preprocessing import StandardScaler
X_scaled = StandardScaler().fit_transform(X)
```

## 2.5. Programación en scikit-learn (explicada parámetro por parámetro)

```
from sklearn.cluster import KMeans

km = KMeans(
    n_clusters=4,          # K: numero de clusters
    init="k-means++",      # inicializacion inteligente
    n_init=10,             # repite con distintas semillas y elige la mejor
    max_iter=300,          # max iteraciones por corrida
    random_state=0         # reproducibilidad
)

labels = km.fit_predict(X_scaled)
centroids = km.cluster_centers_
inertia = km.inertia_
```

### Qué hace cada parte

- `fit_predict`: ajusta (encuentra centroides) y devuelve la etiqueta (cluster) de cada punto.
- `cluster_centers_`: centroides finales (en espacio escalado si escalaste).
- `inertia_`: suma de distancias cuadradas intra-cluster (siempre baja si subes $K$).
- `n_init`: clave para evitar soluciones malas; cuanto mayor, más robusto (pero más coste).

## 2.6. Ejemplo numérico a mano (para entender "cómo asigna")

Datos 1D: $[1, 2, 8, 9]$, $K = 2$.

1. Inicializa centroides: $\mu_1 = 1$, $\mu_2 = 9$.
2. Asignación: $1, 2 \to \mu_1$; $8, 9 \to \mu_2$.
3. Actualización: $\mu_1 = (1 + 2)/2 = 1{,}5$, $\mu_2 = (8 + 9)/2 = 8{,}5$.
4. Repite: no cambia $\Rightarrow$ converge.

**Conclusión**: "predice" asignando cada punto al centroide más cercano.

## 2.7. Cómo elegir $K$ (y por qué el codo no es magia)

**Método del codo (Elbow)** Entrenas varios K y miras `inertia_`. Buscas un punto donde la mejora marginal cae.

```
inertias = []
for k in range(2, 11):
    km = KMeans(n_clusters=k, init="k-means++", n_init=10, random_state=0)
    km.fit(X_scaled)
    inertias.append(km.inertia_)
```

**Cuidado**: siempre baja con $K$. El "codo" a veces no es claro.

**Silhouette score** Mide cohesión y separación. Suele ser más informativo que solo inercia:

```
from sklearn.metrics import silhouette_score
score = silhouette_score(X_scaled, labels)
```

### 2.8. Interpretación de clusters (customer segmentation)

Una práctica típica:

1. Obtienes `labels`.
2. Los añades al DataFrame.
3. Resumes por cluster (medias, medianas, conteos).

```
import pandas as pd

df["cluster"] = labels
summary = df.groupby("cluster").mean(numeric_only=True)
counts = df["cluster"].value_counts()
```

### 2.9. Limitaciones típicas

- Prefiere clusters "esféricos" (redondos) en Euclídea.
- Sensible a outliers.
- Hay que fijar $K$.
- Si hay categorías (texto), hay que codificarlas antes (one-hot, embeddings, etc.).

### 2.10. Pregunta típica: "¿se combinan métodos o van separados?"

En la práctica se combinan MUCHO:

- **PCA + K-Means**: reduces ruido/colinealidad y luego clusterizas.
- **UMAP/t-SNE para visualizar**, pero el clustering lo haces en el espacio original o en PCA.

## 3. DBSCAN (clustering por densidad + detección de ruido)

### 3.1. Idea central

DBSCAN agrupa regiones densas y etiqueta como **ruido** los puntos aislados. Define:

- **Core point**: tiene al menos `min_samples` vecinos en radio `eps`.
- **Border point**: está cerca de un core, pero no cumple densidad.
- **Noise**: no pertenece a ningún cluster (label $-1$).

### 3.2. Parámetros clave

- `eps` ($\varepsilon$): radio de vecindad.
- `min_samples`: mínimo de vecinos para ser core.
- `metric`: distancia (euclídea; en geo conviene haversine).

### 3.3. Algoritmo paso a paso

1. Para cada punto $x_i$ calcula su vecindad $N_\varepsilon(x_i)$.
2. Si $|N_\varepsilon(x_i)| \geq$ `min_samples`, $x_i$ es core.
3. Conecta cores alcanzables por densidad $\Rightarrow$ forma clusters.
4. Asigna border points a clusters cercanos.
5. Los que no entran $\Rightarrow$ ruido ($-1$).

### 3.4. Programación en scikit-learn

```
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler

X_scaled = StandardScaler().fit_transform(X)

db = DBSCAN(
    eps=0.5,
    min_samples=5,
    metric="euclidean"
)

labels = db.fit_predict(X_scaled)
```

#### Interpretación inmediata del output

- `-1` = ruido/outliers.
- `0,1,2,...` = clusters encontrados.

### 3.5. Cómo elegir `eps` (técnica típica)

Una técnica habitual es la **k-distance plot**: ordenas distancias al $k$-ésimo vecino (con $k =$ `min_samples`) y buscas un "codo". (En prácticas, muchas veces basta con probar una rejilla de `eps` y evaluar silhouette ignorando ruido.)

### 3.6. Ejemplo numérico a mano (mini)

Supón puntos en 1D: $[0, 0{,}1, 0{,}2, 5, 5{,}1, 20]$. Con `eps=0.25`, `min_samples=3`:

- $0, 0{,}1, 0{,}2$ forman un cluster (cada uno tiene suficientes vecinos en radio).
- $5, 5{,}1$ no llegan a 3 vecinos $\Rightarrow$ podrían ser ruido o cluster pequeño según parámetros.
- 20 claramente ruido.

Esto ilustra lo esencial: DBSCAN depende muchísimo de `eps` y de densidad.

### 3.7. DBSCAN en geolocalización (lat/long): el detalle importante

Si trabajas con latitud/longitud:

- Euclídea sobre grados puede ser engañosa.
- Una alternativa sólida es usar **Haversine** (distancia sobre esfera).

```
import numpy as np
from sklearn.cluster import DBSCAN

coords = df[["Latitude","Longitude"]].to_numpy()
coords_rad = np.radians(coords)

kms_per_radian = 6371.0088
eps_km = 50  # ejemplo: 50 km
db = DBSCAN(
    eps=eps_km / kms_per_radian,
    min_samples=5,
    metric="haversine"
)

labels = db.fit_predict(coords_rad)
```

### 3.8. Ventajas y limitaciones

#### Ventajas

- No necesitas fijar $K$.
- Encuentra clusters con formas arbitrarias.
- Detecta outliers (ruido).

#### Limitaciones

- Un único `eps` falla si hay **densidades muy distintas**.
- Puede etiquetar demasiado como ruido si ajustas mal.

### 3.9. Pregunta típica: "¿cómo asigno un punto nuevo?"

K-Means sí tiene `predict`. DBSCAN **no** tiene una predicción estándar para puntos nuevos. Opciones prácticas:

- Re-entrenar con el nuevo punto (coste, pero coherente).
- Aproximar: asignar al cluster del core más cercano si cae dentro de `eps`; si no, ruido.
- Entrenar un clasificador (KNN) sobre puntos ya etiquetados para aproximar la asignación.

## 4. HDBSCAN (densidad jerárquica, más robusto)

### 4.1. Por qué existe

DBSCAN necesita un `eps` global. Si hay regiones densas (ciudad) y regiones dispersas (rural), un solo `eps` no vale. **HDBSCAN** construye una jerarquía por densidad y extrae clusters **estables**.

### 4.2. Parámetros clave

- `min_cluster_size`: tamaño mínimo para considerar un cluster.
- `min_samples`: controla la "estrictitud"; si es `None` suele depender de `min_cluster_size`.
- `metric`: distancia.

### 4.3. Programación (librería hdbscan)

```
import hdbscan
from sklearn.preprocessing import StandardScaler

X_scaled = StandardScaler().fit_transform(X)

hdb = hdbscan.HDBSCAN(
    min_cluster_size=10,
    min_samples=None,
    metric="euclidean"
)

labels = hdb.fit_predict(X_scaled)  # ruido = -1
```

### 4.4. Interpretación

- Igual que DBSCAN: `-1` es ruido.
- Suele generar clusters más razonables cuando hay densidad variable.

### 4.5. Ventajas y limitaciones

- **Ventaja**: menos dependencia de un `eps` único.
- **Limitación**: es más complejo, y la interpretación fina requiere entender estabilidad.

# 5. PCA (Análisis de Componentes Principales)

## 5.1. Qué es y qué NO es

**PCA** es reducción de dimensionalidad **lineal**. No clusteriza. Construye nuevas variables (componentes principales) como combinaciones lineales de las originales.

## 5.2. Objetivo

Ordenar componentes para capturar varianza:

$$\mathrm{PC}_1 \text{ maximiza varianza}, \quad \mathrm{PC}_2 \perp \mathrm{PC}_1, \text{ maximiza varianza restante}, \ldots$$

La varianza explicada por cada componente se resume con `explained_variance_ratio_`.

## 5.3. Por qué hay que escalar (casi siempre)

PCA depende de varianzas. Si una variable domina por escala, PC1 se orienta hacia esa variable. Solución:

```
from sklearn.preprocessing import StandardScaler
X_scaled = StandardScaler().fit_transform(X)
```

## 5.4. Programación en scikit-learn (y qué significa cada atributo)

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2, random_state=0)
Z = pca.fit_transform(X_scaled)

components = pca.components_                # loadings (direcciones)
evr = pca.explained_variance_ratio_         # varianza explicada por
    componente
```

**`components_` (loadings)** Es una matriz (n_components × n_features). Cada fila es un vector dirección:

- PC1 = `components_[0]`
- PC2 = `components_[1]`

**Interpretación práctica** Si en PC1 pesan mucho `petal length` y `petal width`, PC1 representa "tamaño del pétalo".

## 5.5. Elegir número de componentes

Tres enfoques:

1. Fijar número: `PCA(n_components=2)` para visualizar.

2. Por varianza: `PCA(n_components=0.95)` conserva 95 %.

3. Analizar scree plot: barras de `explained_variance_ratio_` y acumulada.

### 5.6. Ejemplo numérico conceptual (2D correlacionado)

Si tus datos forman una nube diagonal, PC1 se alinea con esa diagonal y PC2 con la perpendicular. La proyección se calcula como producto escalar:

$$z_{i,1} = x_i \cdot w_1, \quad z_{i,2} = x_i \cdot w_2$$

donde $w_1, w_2$ son los vectores de `components_`.

### 5.7. PCA + clustering (combinación típica)

- PCA reduce ruido y colinealidad.
- K-Means mejora en alta dimensión (menos dimensiones basura).

Pipeline recomendado:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("pca", PCA(n_components=0.95, random_state=0)),
    ("kmeans", KMeans(n_clusters=4, init="k-means++", n_init=10,
       random_state=0))
])

labels = pipe.fit_predict(X)
```

## 6. t-SNE (visualización preservando vecindades locales)

### 6.1. Qué hace

t-SNE construye un embedding 2D/3D preservando **vecindades locales**: si dos puntos son vecinos en alta dimensión, intenta que lo sigan siendo en el embedding.

### 6.2. Cuándo usarlo

- Visualizar estructura de datos de alta dimensión (texto, imágenes, perfiles).
- Explorar separaciones y subgrupos.

### 6.3. Lo que NO debes interpretar

- Los ejes X/Y no tienen significado interpretativo como en PCA.
- Distancias globales entre clusters pueden estar distorsionadas.

### 6.4. Parámetros clave y su significado

- `perplexity`: tamaño efectivo del vecindario (5–50 típico).
- `init`: `"pca"` suele ser más estable que `random"`.
- `learning_rate`, `max_iter`: afectan convergencia.
- `random_state`: reproducibilidad.

### 6.5. Programación recomendada (con PCA previo)

t-SNE puede ser costoso; se usa PCA previo a 30–50 dims:

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

X_scaled = StandardScaler().fit_transform(X)
X_pca50 = PCA(n_components=50, random_state=0).fit_transform(X_scaled)

tsne = TSNE(
    n_components=2,
    perplexity=30,
    init="pca",
    max_iter=1000,
    learning_rate="auto",
    random_state=0
)

Z = tsne.fit_transform(X_pca50)
```

### 6.6. Cómo se usa con clustering

Dos usos habituales:

- (A) Clusterizas en $X$ (o en PCA) y luego pintas en t-SNE para **ver** clusters.
- (B) Pintas t-SNE para explorar y luego decides método/parametrización.

**Recomendación**: no confundir “se ve separado en t-SNE” con “es separable en el espacio real”.

# 7. UMAP (embeddings rápidos y transformables)

## 7.1. Qué hace

UMAP construye un embedding preservando estructura local (y parte de global) mediante un grafo de vecinos.

## 7.2. Por qué es tan usado

- Suele escalar mejor que t-SNE.
- Permite `transform` de nuevos datos tras entrenar.

## 7.3. Parámetros clave

- `n_neighbors`: local vs global (bajo = más local, alto = más global).
- `min_dist`: compacidad de clusters en el embedding (bajo = más compacto).
- `metric`: euclidean, cosine, etc.
- `random_state`: reproducibilidad.

## 7.4. Programación

```
from sklearn.preprocessing import StandardScaler
import umap

X_scaled = StandardScaler().fit_transform(X)

um = umap.UMAP(
    n_components=2,
    n_neighbors=15,
    min_dist=0.1,
    metric="euclidean",
    random_state=0
)

Z = um.fit_transform(X_scaled)
```

## 7.5. UMAP con datos nuevos (ventaja clave)

```
um = umap.UMAP(random_state=0).fit(X_train_scaled)
Z_train = um.transform(X_train_scaled)
Z_test  = um.transform(X_test_scaled)
```

# 8. Comparativa final (qué método usar, cuándo y por qué)

| Método | Cuándo usar | Ventajas | Puntos críticos |
|---|---|---|---|
| K-Means | Clusters "redondos", segmentación clásica | Rápido, simple, interpretable por centroides | Elegir $K$, escalado, outliers |
| DBSCAN | Formas arbitrarias + outliers + sin $K$ | Encuentra ruido, no fija $K$ | Elegir `eps`, densidad variable |
| HDBSCAN | Densidad variable (urbano/rural) | Más robusto que DBSCAN | Parámetros de tamaño/estrictitud |
| PCA | Reducir/visualizar, quitar colinealidad | Lineal e interpretable (loadings) | Escalado, no clusteriza |
| t-SNE | Visualización local de alta dimensión | Muy bueno para "ver" grupos | No interpretes ejes, sensible, coste |
| UMAP | Visualización rápida y transformable | Rápido, `transform`, flexible | `n_neighbors`, `min_dist` |

# 9. Casos de uso reales (empresa / vida real) y cómo se conectan con el código

## 9.1. Segmentación de clientes (K-Means / HDBSCAN)

**Objetivo**: crear grupos para marketing o pricing.

- Variables: frecuencia compra, gasto medio, antigüedad, ingresos, etc.
- Preproceso: `StandardScaler`.
- Modelo: K-Means si esperas clusters compactos; HDBSCAN si hay densidades distintas y outliers.
- Interpretación: `groupby(cluster).mean()` + conteos + reglas de acción.

## 9.2. Clustering geográfico (DBSCAN/HDBSCAN)

**Objetivo**: detectar zonas con alta concentración (tiendas, incidencias, museos).

- Usar distancia Haversine si trabajas con lat/long.
- Ruido ($-1$) = ubicaciones aisladas que no forman zona.
- Acción: optimizar rutas, ubicar puntos de servicio, detectar "hotspots".

## 9.3. Visualización de datos complejos (PCA, t-SNE, UMAP)

**Objetivo**: entender estructura y preparar decisiones.

- PCA para resumen e interpretación (qué variables explican la variabilidad).
- UMAP/t-SNE para inspección visual de subgrupos.
- Acción: detectar segmentos ocultos, ver mezclas, validar features.

## 10. Checklist de examen/prácticas (lo que suele hacerte perder puntos)

- No escalar cuando toca (K-Means / DBSCAN / UMAP / t-SNE).
- Meter IDs como features (CustomerID, etc.) y contaminar clustering.
- No interpretar labels: no basta con sacar clusters, hay que describir perfiles.
- Confundir PCA/t-SNE/UMAP con clustering (son reducción/visualización).
- No mencionar limitaciones (K-Means esférico, DBSCAN densidad única, etc.).
- No explicar hiperparámetros (K, eps, min_samples, min_cluster_size, perplexity, n_neighbors...).

## 11. Resumen final ultra-conciso (por si quieres memorizar)

- **K-Means**: centroides, necesitas $K$, rápido, clusters compactos, escalar.
- **DBSCAN**: densidad, `eps`/`min_samples`, ruido $-1$, formas raras, densidad variable = problema.
- **HDBSCAN**: DBSCAN jerárquico, mejor con densidad variable, `min_cluster_size`.
- **PCA**: lineal, interpretable por loadings, reduce dimensión, escalar, varianza explicada.
- **t-SNE**: vecindades locales, visualización, ejes no interpretables, sensible.
- **UMAP**: visualización rápida, `transform`, controla local/global con vecinos y distancia mínima.